

A Comparative Analysis of Few-Shot and Per-Scene NeRF models on Sparse-View Datasets

Abstract—This work compares Generalizable NeRFs and Fast Optimization Per-Scene trained models applied to sparse-view datasets, evaluating the impact of these feature extraction architectures on novel view synthesis performance. Through a quantitative analysis using visual fidelity (PSNR, SSIM, LPIPS) and computational efficiency metrics (convergence time, VRAM footprint, inference time), this study highlights the trade-offs between the global generalization of Transformers and the local processing efficiency of convolutions, as well as the critical impact of representational collapse in few-shot models. Ultimately, it aims to guide the selection of 3D rendering models for practical scenarios often constrained by hardware limitations and data scarcity.

I. INTRODUCTION

Neural Radiance Fields (NeRFs) have consolidated themselves in recent years as the state-of-the-art for novel view synthesis, in both industry and academia [1]–[3]. However, the most advanced neural rendering techniques demand vast amounts of: (i) computational power; and (ii) multiple views in the training dataset. Both challenges make practical applications, such as mobile scanning, virtual try-ons, and synthesis in reduced datasets, unfeasible. To address this problem, projects such as Instant-NGP (*Instant Neural Graphics Primitives*), Nerfacto, PixelNeRF, and GNT (*Generalizable NeRF Transformer*) [2], [4]–[6], among others, attempt to expand the state-of-the-art and, consequently, tackle analogous themes, such as: (i) spatial parameter *encoding* [4]; (ii) the use of hybrid architectures combining Convolutional Neural Networks (CNNs) and Vision Transformers (ViT) [6]–[8]; and (iii) the pre-computation (*baking*) of models into structures with a smaller memory footprint [9].

Consequently, this work aims to explore the use of different neural radiance field models and their respective optimization techniques by applying them to sparse-view datasets, promoting an exploratory analysis between computational performance and the visual quality of the generated volumes. To evaluate reconstruction fidelity, visual quality metrics widely established in the literature will be used, such as PSNR (*Peak Signal-to-Noise Ratio*), SSIM (*Structural Similarity Index Measure*) [10], and LPIPS (*Learned Perceptual Image Patch Similarity*) [11]. In parallel, the quantification of operational costs will be based on systemic performance metrics, including convergence time during training, inference speed, and memory *footprint*, especially VRAM (*Video Random Access Memory*) consumption. Although works like [12] and [13] have evaluated NeRFs in limited data regimes, and projects like NerfBaselines [14] have reported results on traditional and per-scene training-based NeRFs, a systematic comparison

between generalizable models with convolutional extractors versus attention-based ones, under controlled hardware conditions and reproducible splits, remains unexplored in the literature. In short, this investigation provides a consolidated panorama of the *trade-offs* between computational efficiency and visual fidelity, establishing clear guidelines for the viability and practical application of novel view synthesis models in highly constrained scenarios, particularly regarding data and hardware limitations.

Given this problem, the contributions of this work are: (i) a systematic and reproducible evaluation of few-shot and per-scene NeRF models under three data regimes; and (ii) the public release of integration wrappers for PixelNeRF and GNT into the Nerfstudio framework at <https://github.com/felipe-gsilva/few-shot-nerfs>, adapting their input and output interfaces to comply with the framework’s unified evaluation protocol, without modifying the pre-trained weights or original inference logic, to allow reproducible comparisons under controlled conditions.

II. MATERIALS AND METHODS

This section outlines the methodology adopted for the experimental phase of this work. Figure 1 illustrates the proposed execution protocol and details the isolated environment configuration. To systematically evaluate the trade-offs between visual fidelity and computational efficiency, the subsequent steps of this study are organized as follows: (i) the selection and sampling strategy of the dataset; (ii) the chosen neural rendering architectures and their respective training regimes; and (iii) the specific criteria and metrics used for evaluation.

A. Dataset & Splits

The experiments were conducted on the LLFF (*Local Light Field Fusion*) dataset [15], which comprises 8 real-world scenes (*fern*, *flower*, *fortress*, *horns*, *leaves*, *orchids*, *room*, *trex*) captured with a forward-facing moving camera. Camera poses and depth bounds were obtained via COLMAP, following standard dataset procedures. Furthermore, images were evaluated across different resolutions, specifically downscale 4 (1008×756) and downscale 8 (504×378).

The complete data splitting protocol follows the few-shot evaluation configuration established by Niemeyer et al. [?]. Specifically, the testing split was generated by reserving images with indices that are multiples of 8 (`11ffhold=8`), while the training views for the sparse regimes were strictly limited to $N \in \{3, 6, 10\}$. The selection of these training views was performed using two distinct strategies: (i) *uniform*, where

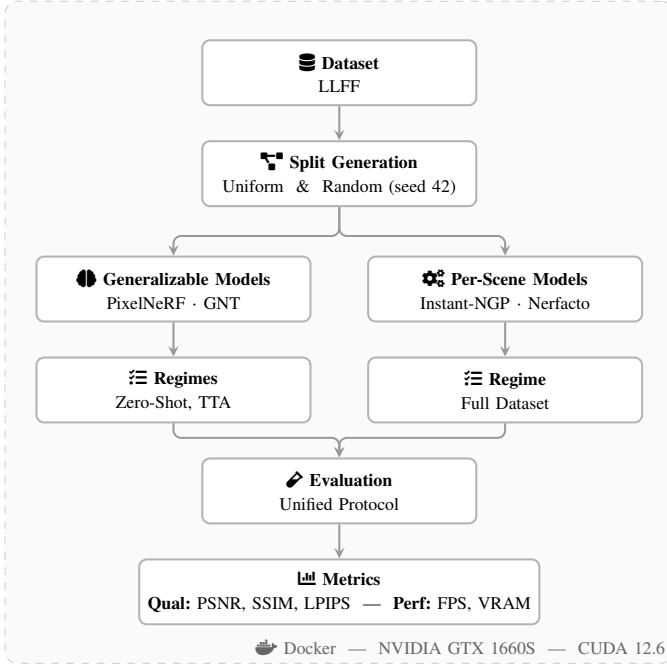


Fig. 1: Proposed methodology pipeline. The flowchart details the progression from data preparation to the bifurcated training paradigms, culminating in a unified evaluation protocol. The entire pipeline executes within an isolated containerized environment.

TABLE I: Impact of the training view selection strategy (*uniform* vs. *random*) on average visual quality. Evaluated under the TTA regime with $N = 10$ views on the *fern* scene.

Model	Split	PSNR (dB) $\uparrow$	SSIM $\uparrow$	LPIPS $\downarrow$	Δ PSNR
PixelNeRF	Uniform	6.98	0.047	0.846	+0.54
	Random	6.44	0.042	0.840	
GNT	Uniform	10.34	0.179	0.738	+0.15
	Random	10.19	0.184	0.742	
Instant-NGP	Uniform	13.57	0.231	0.627	+0.06
	Random	13.51	0.229	0.630	
Nerfacto	Uniform	12.04	0.227	0.809	+0.11
	Random	11.93	0.226	0.865	

views are sampled via linear interpolation over the dataset indices, distributing them evenly along the camera trajectory (a strategy that does not generalize to other datasets, since LLFF indices intrinsically encode camera trajectory order); and (ii) *random*, where views are randomly selected using a fixed seed (42) to ensure reproducibility. Comparing these two strategies allows us to evaluate the models’ sensitivity to the spatial distribution of the input views, as demonstrated in Table I. Given the empirical advantage of uniform sampling (+0.54 dB PSNR on the *fern* scene), this strategy was adopted for all subsequent experiments.

Beyond the core optimization hyperparameters, specific pipeline configurations were strictly enforced to guarantee stability and computational efficiency across the few-shot regimes. Notably, camera pose refinement was disabled for

Nerfacto to prevent the catastrophic geometric collapse frequently observed in sparse-view scenarios. To minimize memory footprint and accelerate the Test-Time Adaptation phase, mixed-precision training was enabled across the relevant architectures. For the hash-based models (Instant-NGP and Nerfacto), the multiscale hash table capacity was constrained to a maximum size of 2^{16} , with backgrounds explicitly set to white. Furthermore, structural hyperparameters for GNT were locked to 48 coarse and importance samples per ray, a transformer depth of 8, and a network width of 64, ensuring reproducible structural bounds during volume rendering.

B. Models & Training Regimes

This study evaluates representative models for novel view synthesis under few-shot constraints, encompassing distinct architectural paradigms. The generalizable approaches selected are: (i) **PixelNeRF** [5], which utilizes Convolutional Neural Networks (CNNs) to extract spatial features and inject them into the renderer via projection; and (ii) **Generalizable NeRF Transformer (GNT)** [6], which replaces both the feature extractor and the differentiable renderer with cross-attention modules. As comparative baselines, two highly efficient *per-scene* architectures were chosen: **Instant-NGP** [4], which replaces traditional MLPs with a multiscale hash table encoding, and **Nerfacto** [2], an aggregate model combining state-of-the-art heuristics for rapid convergence and high fidelity.

The experimental execution was structured across three distinct regimes: (i) **Zero-Shot**: The ideal paradigm for extreme compute constraints, performing instant inference using pre-trained weights without any local optimization on the novel scene; (ii) **Test-Time Adaptation (TTA)**: An intermediate regime where the pre-trained generalizable models are fine-tuned using the sparse input views ($N \in \{3, 6, 10\}$); (iii) **Per-Scene (100%)**: Representing the upper bound for visual fidelity, this regime optimizes the baseline models from scratch using the entirety of available images for a specific scene. It is crucial to note that during the Test-Time Adaptation (TTA) phase for the generalizable models, the core feature extraction networks (CNN and ViT) were kept frozen, updating only the terminal rendering layers. Preliminary ablation tests revealed that unfreezing the entire network under extreme data sparsity ($N \leq 10$) led to catastrophic forgetting and severe overfitting to the training poses, actively degrading novel view synthesis performance.

To ensure a strictly equitable evaluation protocol, all models were unified under the *Nerfstudio* framework [2]. Instant-NGP and Nerfacto are natively supported by the framework. Conversely, integration wrappers were developed for PixelNeRF and GNT, porting their original DataManager and Pipeline components to comply with Nerfstudio’s modular interface. This unified environment leverages *tiny-cuda-nn* [16] for GPU acceleration and standardizes the logging infrastructure.

For consistency across the evaluation, all models were optimized using Adam with a standard learning rate of $\eta = 10^{-3}$ and zero weight decay. While the original GNT and PixelNeRF literature suggests slightly lower learning rates (between 10^{-4}

TABLE II: Training and optimization hyperparameters across evaluated regimes. TTA configurations denote the fine-tuning phase.

Model	Regime	Iterations (Steps)	Rays / Batch	Learning Rate (η)
PixelNeRF	TTA	2500	512	10^{-3}
	Zero-Shot	0	512	N/A
GNT	TTA	2500	512	10^{-3}
	Zero-Shot	0	512	N/A
Instant-NGP	Per-Scene	30000	512	10^{-3}
Nerfacto	Per-Scene	30000	512	10^{-3}

and 5×10^{-4}), adopting $\eta = 10^{-3}$ strictly standardizes the hyperparameter baseline across all architectures a criterion that factors into the subsequent TTA stability analysis, as can be seen in Table II. The complete wrapper integration code is publicly available at <https://github.com/Felipe-gsilva/Few-Shot-NeRFs/>. Additionally, the modular wrappers can be downloaded directly as packages from PyPI and piwheels (<https://www.piwheels.org/project/nerfstudio-gnt/> and <https://www.piwheels.org/project/nerfstudio-pixelnerf/>), while the reproducible experiment scripts and environment configurations are hosted at <https://github.com/Felipe-gsilva/nerf-ann-paper/>.

C. Evaluation & Metrics

The evaluation framework assesses three complementary dimensions: visual fidelity, computational efficiency, and memory consumption. Final metrics were computed exclusively on the held-out test set (`fixed_indices_eval_dataloader`), ensuring no overlap with the training views.

To quantify visual quality, this project used three standard novel view synthesis metrics: (i) **PSNR** (Peak Signal-to-Noise Ratio), to measure pixel-wise fidelity; (ii) **SSIM** (Structural Similarity Index Measure) [10], to evaluate structural coherence; and (iii) **LPIPS** (Learned Perceptual Image Patch Similarity) [11]. Following the protocol established by Niemeyer et al. [?], LPIPS was computed using Visual Geometry Group (VGG) features to accurately quantify perceptual distance.

Operational efficiency was measured through the inference rendering rate in frames per second (fps) across the test cameras. For memory evaluation, both the static parameter footprint of the models and the dynamic VRAM allocation peak logged by PyTorch during runtime were analyzed.

To guarantee environmental isolation and reproducibility, all experiments were executed within Nvidia Docker containers, preventing external processes from adding noise to the performance metrics. The hardware setup consisted of an NVIDIA GeForce GTX 1660 Super (6GB VRAM) running CUDA 12.6.

III. RESULTS AND DISCUSSION

This section presents the consolidated quantitative, computational and qualitative analyses across the evaluated LLFF scenes. Table VI summarizes the global averages for visual fidelity, temporal efficiency, and memory footprint. The results

reveal critical trade-offs between generalization, optimization stability, and hardware constraints.

A. Visual Fidelity and Architectural Inertia

The per-scene architectures demonstrated vastly superior visual reconstruction capabilities. Nerfacto established the performance upper bound, achieving a global average of 27.11 dB PSNR and 0.875 SSIM. In contrast, the generalizable models struggled significantly with the sparse constraints of the LLFF dataset.

A notable phenomenon observed in the consolidated data is the performance degradation during Test-Time Adaptation (TTA). Counterintuitively, fine-tuning the generalizable models on sparse views slightly decreased their PSNR compared to their Zero-Shot baselines. PixelNeRF dropped from 6.62 dB to 6.38 dB, while GNT decreased from 10.73 dB to 10.42 dB. Although numerically modest, this degradation has substantial interpretive impact, suggesting a representational collapse or "architectural inertia", where the lack of dense geometric priors and depth regularization during few-shot fine-tuning causes the networks to overfit the sparse inputs, actively destroying the generalized prior they possessed in the Zero-Shot regime.

TABLE III: Visual Quality on the *fern* scene.

Regime	Model	PSNR (dB) $\uparrow$	SSIM $\uparrow$	LPIPS $\downarrow$
Zero-Shot	GNT	10.26	0.290	0.732
TTA	GNT	10.26	0.290	0.732
Zero-Shot	PixelNeRF	5.74	0.093	0.832
TTA	PixelNeRF	6.07	0.058	0.812
Per-Scene	Instant-NGP	13.41	0.433	0.497
Per-Scene	Nerfacto	25.75	0.850	0.184

TABLE IV: Impact of the number of training views (N) on PSNR (dB) for generalizable models on the *fern* scene.

Model	$N = 3$	$N = 6$	$N = 10$
GNT (Zero-Shot)	10.33	10.19	10.26
GNT (TTA)	10.33	10.19	10.26
PixelNeRF (Zero-Shot)	5.68	5.76	5.79
PixelNeRF (TTA)	6.07	6.05	6.09

B. Computational Trade-offs

From a systemic viability perspective, a clear dichotomy emerges between inference latency and training overhead. While generalizable models bypass extensive optimization (requiring only seconds for Zero-Shot inference or ~ 7 minutes for TTA), their volumetric feature aggregation mechanisms result in catastrophic inference bottlenecks, yielding rendering speeds below 0.02 FPS.

Conversely, the explicit multiscale hash representations of the per-scene models proved vastly more efficient during runtime. Despite requiring lengthier convergence periods (from

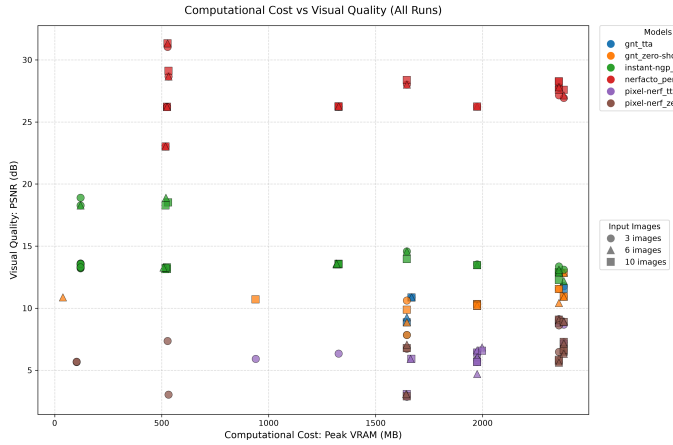


Fig. 2: Trade-off between visual fidelity (PSNR) and computational cost (Peak VRAM) across all evaluated scenes and regimes. The top-left quadrant represents the ideal operational region (high quality, low memory footprint), which is heavily dominated by per-scene hash-based architectures.

~9 minutes for Nerfacto up to ~32 minutes for Instant-NGP), they achieve inference rates up to $97\times$ faster than generalizable approaches (0.388 FPS). Furthermore, the hash-based models exhibit a highly optimized dynamic memory footprint, with Instant-NGP operating at just 1.09 GB of VRAM during inference, solidifying per-scene methods as the most viable path for hardware-constrained deployments despite the initial training cost.

TABLE V: Computational Performance and Memory on the *fern* scene.

Regime	Model	Train Time (s) ↓	Inference (FPS) ↑	VRAM Peak (MB) ↓
Zero-Shot	GNT	8.77	0.007	1974.22
TTA	GNT	392.15	0.008	1974.22
Zero-Shot	PixelNeRF	39.02	0.003	1605.10
TTA	PixelNeRF	496.82	0.006	1924.02
Per-Scene	Instant-NGP	1707.91	0.183	764.74
Per-Scene	Nerfacto	423.45	0.371	1101.83

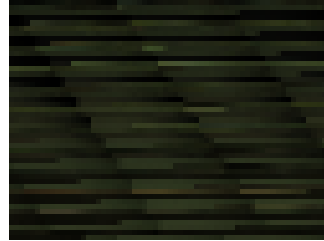
C. Qualitative Analysis

The qualitative assessment visually corroborates the quantitative metrics, specifically illustrating the boundaries of each architectural paradigm. For visual clarity, we synthesized novel views matching the exact poses from the test set using perspective projection.

Figure 3 highlights the severe disparities in reconstruction quality on the highly detailed *fern* scene. As visually evident, neither PixelNeRF nor GNT achieved proper convergence. PixelNeRF produced renderings with intense isotropic blurring, a typical artifact indicating the network’s failure to accurately localize 2D projected features across extremely sparse views. GNT, on the other hand, demonstrated a complete generalization failure regarding the *in-the-wild* characteristics of the scene. This resulted in a representational visual collapse, heavily distorting the volumetric geometry into noise, which persisted even after the 10-view Test-Time Adaptation (TTA).



Ground Truth



PixelNeRF (TTA 10)



GNT (TTA 10)



Instant-NGP (Per-Scene)



Nerfacto (Per-Scene)

Fig. 3: Qualitative comparison on the *fern* scene. Generalizable architectures (PixelNeRF and GNT) failed to converge under sparse constraints, resulting in severe blurring and visual collapse. In contrast, per-scene optimization models (Instant-NGP and Nerfacto) successfully reconstructed the complex geometry.

Conversely, the per-scene architectures successfully captured the intricate foliage geometry. While Instant-NGP exhibited minor volumetric density fluctuations (floaters) in sparsely covered peripheral regions, Nerfacto produced a highly faithful reconstruction, preserving high-frequency details directly matching the native resolution of the Ground Truth.

IV. CONCLUSION

In this paper, we explored the pros and cons of the usage of Per-Scene and Few-Shot NeRFs in order to guide the selection of 3D rendering models in real-world and scientific applications with hardware and data scarcity constraints. It was observed that per-scene architectures are the unequivocal choice whenever inference latency and VRAM are the binding constraints, given their order-of-magnitude advantage in both axes. However, this conclusion does not generalize to acquisition-constrained domains: per-scene models require dense multi-view capture that is often physically infeasible (e.g., histology, holotomography), a scenario where generaliz-

TABLE VI: Consolidated quantitative results across the LLFF dataset. Metrics represent the global average. Training time reflects the fine-tuning phase for TTA and full convergence for Per-Scene models. VRAM Peak denotes dynamic memory allocation during inference.

Paradigm	Model & Regime	PSNR (dB) $\uparrow$	SSIM $\uparrow$	LPIPS $\downarrow$	Train / TTA Time	Inference (FPS) $\uparrow$	VRAM Peak (GB) $\downarrow$
Generalizable	PixelNeRF (Zero-Shot)	6.62	0.093	0.814	~ 22 s	0.004	1.93
	PixelNeRF (TTA)	6.38	0.071	0.801	4 min 23 s	0.006	1.97
	GNT (Zero-Shot)	10.73	0.300	0.760	~ 9 s	0.014	1.94
	GNT (TTA)	10.42	0.336	0.724	7 min 19 s	0.015	1.93
Per-Scene	Instant-NGP (100%)	14.06	0.442	0.498	31 min 51 s	0.199	1.09
	Nerfacto (100%)	27.11	0.875	0.171	8 min 57 s	0.388	1.43

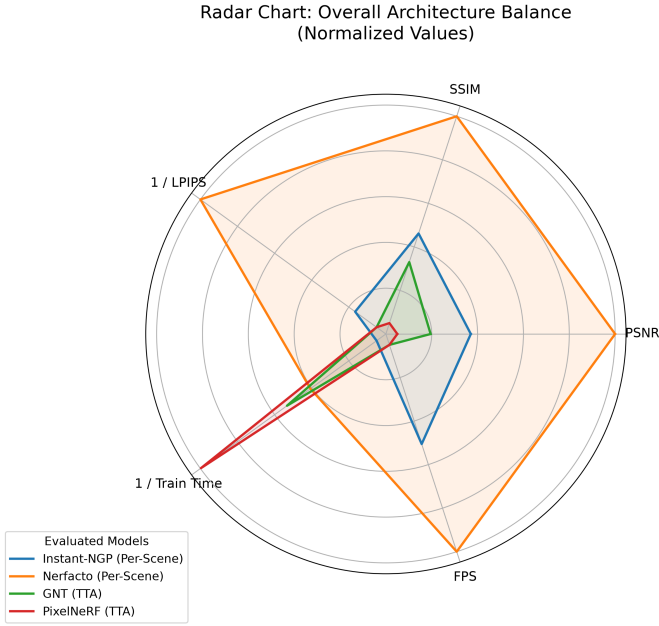


Fig. 4: Radar chart comparing overall architecture balance.

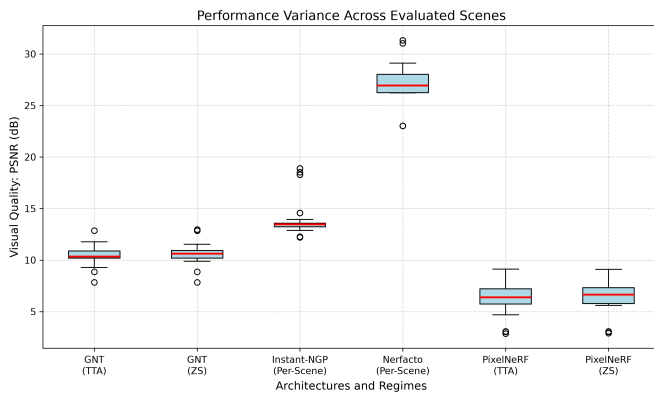


Fig. 5: Performance variance across evaluated scenes.

able models remain the only viable option despite their current fidelity ceiling.

While this study provides a consolidated analysis of novel view synthesis under sparse constraints, we acknowledge

certain limitations. The strict hardware constraints of our isolated environment (6GB VRAM) precluded the evaluation of architectures that rely on heavy 3D cost volumes or extensive geometric regularizations, such as RegNeRF. Furthermore, our empirical validation was restricted to the forward-facing LLFF dataset. Future work includes expanding this evaluation protocol to unbounded, 360-degree datasets to verify these architectural trade-offs under complex geometries. Additionally, we intend to investigate the integration of diffusion models as generative priors to mitigate the representational collapse observed in generalizable models during few-shot adaptation.

ACKNOWLEDGMENT

The authors would like to thank State of São Paulo University and the Interdisciplinary Image Processing and Analysis Laboratory for providing the infrastructure necessary to conduct this research.

REFERENCES

- [1] B. Kerbl, G. Kopanas, T. Leimkühler, and G. Drettakis, “3d gaussian splatting for real-time radiance field rendering,” *ACM Transactions on Graphics*, vol. 42, no. 4, July 2023. [Online]. Available: <https://repo-sam.inria.fr/fungraph/3d-gaussian-splatting/>
- [2] M. Tancik, E. Weber, E. Ng, R. Li, B. Yi, T. Wang, A. Kristoffersen, J. Austin, K. Salahi, A. Ahuja, D. Mcallister, J. Kerr, and A. Kanazawa, “Nerfstudio: A modular framework for neural radiance field development,” in *Special Interest Group on Computer Graphics and Interactive Techniques Conference Proceedings*, ser. SIGGRAPH ’23. ACM, Jul. 2023, p. 1–12. [Online]. Available: <http://dx.doi.org/10.1145/3588432.3591516>
- [3] Adobe, *Substance 3D Viewer (beta)*, 2025, computer Software. [Online]. Available: <https://helpx.adobe.com/substance-3d-viewer/get-started/quick-start.html>
- [4] T. Müller, A. Evans, C. Schied, and A. Keller, “Instant neural graphics primitives with a multiresolution hash encoding,” *ACM Transactions on Graphics*, vol. 41, no. 4, p. 1–15, Jul. 2022. [Online]. Available: <http://dx.doi.org/10.1145/3528223.3530127>
- [5] A. Yu, V. Ye, M. Tancik, and A. Kanazawa, “pixelnerf: Neural radiance fields from one or few images,” 2021. [Online]. Available: <https://arxiv.org/abs/2012.02190>
- [6] Z. Wang, M. Varma, P. Wang, X. Chen, T. Chen, and S. Venugopalan, “Is attention all that nerf needs?” 2023. [Online]. Available: <https://arxiv.org/abs/2207.13298>
- [7] K. Schwarz, Y. Liao, M. Niemeyer, and A. Geiger, “Graf: Generative radiance fields for 3d-aware image synthesis,” 2021. [Online]. Available: <https://arxiv.org/abs/2007.02442>
- [8] H. Wu, B. Xiao, N. Codella, M. Liu, X. Dai, L. Yuan, and L. Zhang, “Cvt: Introducing convolutions to vision transformers,” in *Proceedings of the IEEE/CVF International Conference on Computer Vision (ICCV)*, October 2021, pp. 22–31.

- [9] P. Hedman, P. P. Srinivasan, B. Mildenhall, J. T. Barron, and P. Debevec, "Baking neural radiance fields for real-time view synthesis," 2021. [Online]. Available: <https://arxiv.org/abs/2103.14645>
- [10] Z. Wang, A. C. Bovik, H. R. Sheikh, and E. P. Simoncelli, "Image quality assessment: from error visibility to structural similarity," *IEEE transactions on image processing*, vol. 13, no. 4, pp. 600–612, 2004.
- [11] R. Zhang, P. Isola, A. A. Efros, E. Shechtman, and O. Wang, "The unreasonable effectiveness of deep features as a perceptual metric," in *Proceedings of the IEEE conference on computer vision and pattern recognition*, 2018, pp. 586–595.
- [12] M. Niemeyer, J. T. Barron, B. Mildenhall, M. S. Sajjadi, A. Geiger, and N. Radwan, "Regnerf: Regularizing neural radiance fields for view synthesis from sparse inputs," in *Proceedings of the IEEE/CVF Conference on Computer Vision and Pattern Recognition*, 2022, pp. 5480–5490.
- [13] H. Heo, T. Kim, J. Lee, J. Lee, S. Kim, H. Kim, and J.-H. Kim, "Robust camera pose refinement for multi-resolution hash encoding," 02 2023.
- [14] J. Kulhanek and T. Sattler, "Nerfbaselines: Consistent and reproducible evaluation of novel view synthesis methods," 2025. [Online]. Available: <https://arxiv.org/abs/2406.17345>
- [15] B. Mildenhall, P. P. Srinivasan, R. Ortiz-Cayon, N. K. Kalantari, R. Ramamoorthi, R. Ng, and A. Kar, "Local light field fusion: Practical view synthesis with prescriptive sampling guidelines," 2019. [Online]. Available: <https://arxiv.org/abs/1905.00889>
- [16] T. Müller, "tiny-cuda-nn," <https://github.com/NVlabs/tiny-cuda-nn>, 2021, version 1.7, BSD-3-Clause License.